Machine Learning

Notes by Marco Galletti
Lecturer: Stephan Günnemann

Technical University of Munich
Winter Semester 2026/27

Contents

1 k-Nearest Neighbors **2**
1.1 The k-NN algorithm . . . 2
1.2 Choosing k and measuring performance . . . 3
1.3 Distances, scaling and dimensionality . . . 4

2 Decision Trees **5**
2.1 Decision trees and inference . . . 5
2.2 Growing a tree: splitting criteria . . . 6
2.3 Overfitting, pruning and extensions . . . 9

1 k-Nearest Neighbors

1.1 The k-NN algorithm

1-NN. Let $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ be a training set with features $\boldsymbol{x}_i \in \mathbb{R}^D$ and class labels $y_i \in \{1, \dots, C\}$. The **1-NN** rule fixes a distance measure d (e.g. Euclidean) and gives a new observation the label of its nearest training point; the same rule does regression by copying a real-valued y_i. 1-NN corresponds to the Voronoi tessellation of the training points: each point labels its own cell, and the decision region of a class is the union of the cells of its points. Every single point, a mislabelled one included, owns a cell, so 1-NN generalises poorly.

k-NN. Letting several neighbours vote and taking the majority makes the rule robust against errors in the training set.

Definition 1.1 (k-NN classifier)**.** Let $\mathcal{N}_k(\boldsymbol{x})$ be (the indices of) the k training points nearest to $\boldsymbol{x}$, and let the **indicator variable** $\mathbb{I}(e)$ be 1 if e is true and 0 otherwise. The **k-NN classifier** predicts

$$p(y = c \mid \boldsymbol{x}, k) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \mathbb{I}(y_i = c), \qquad \hat{y} = \arg\max_c \; p(y = c \mid \boldsymbol{x}, k),$$

i.e. $\boldsymbol{x}$ gets the mode of its neighbours' labels.

For fixed $\boldsymbol{x}$, $p(\cdot \mid \boldsymbol{x}, k)$ is a probability mass function on the classes: it is nonnegative and, since each neighbour has exactly one label ($\sum_{c=1}^C \mathbb{I}(y_i = c) = 1$),

$$\sum_{c=1}^{C} p(y = c \mid \boldsymbol{x}, k) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \sum_{c=1}^{C} \mathbb{I}(y_i = c) = \frac{k}{k} = 1.$$

It is the fraction of each class among the k neighbours, like the fraction of each colour in a jar: with 4 red, 10 green and 6 blue balls, $p(\text{red}) = 4/20 = 0.2$, $p(\text{green}) = 10/20 = 0.5$, $p(\text{blue}) = 6/20 = 0.3$.

Definition 1.2 (Weighted k-NN, k-NN regression)**.** In **weighted k-NN** each neighbour votes with a weight inversely proportional to its distance:

$$p(y = c \mid \boldsymbol{x}, k) = \frac{1}{Z} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \frac{\mathbb{I}(y_i = c)}{d(\boldsymbol{x}, \boldsymbol{x}_i)}, \qquad Z = \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \frac{1}{d(\boldsymbol{x}, \boldsymbol{x}_i)},$$

and again $\hat{y} = \arg\max_c \; p(y = c \mid \boldsymbol{x}, k)$. Dividing by Z makes the weights sum to 1:

$$w_i = \frac{1}{Z \, d(\boldsymbol{x}, \boldsymbol{x}_i)}, \qquad \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} w_i = \frac{Z}{Z} = 1, \qquad p(y = c \mid \boldsymbol{x}, k) = \sum_{i \in \mathcal{N}_k(\boldsymbol{x}) \colon \, y_i = c} w_i,$$

so $p(y = c \mid \boldsymbol{x}, k)$ is the total weight of the neighbours of class c, and again a probability mass function. In **k-NN regression** ($y_i \in \mathbb{R}$) the prediction is the mean of the neighbours' values with the same weights:

$$\hat{y} = \frac{1}{Z} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} \frac{y_i}{d(\boldsymbol{x}, \boldsymbol{x}_i)} = \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} w_i \, y_i.$$

Effect of k. With $k = 1$, an isolated training point lying among points of another class (e.g. a mislabelled one) gets its own small island in the decision regions. With $k = 3$ such a point is outvoted by its two other neighbours, so these islands mostly disappear and the decision boundaries become smoother (fig. 1). How to choose k is the topic of the next subsection.

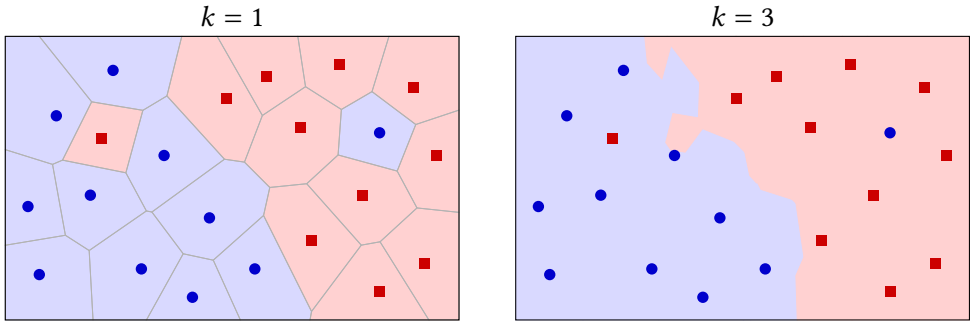


Figure 1: Decision regions of 1-NN (with the Voronoi cells) and 3-NN on the same two-class data.

1.2 Choosing k and measuring performance

Train, validation, test. The number of neighbours k is a **hyper-parameter**. The goal is **generalisation**: pick the k that performs best, in a predefined metric such as accuracy, on unseen future data. Future data are not available, so $\mathcal{D}$ is split (fig. 2) into a **learning set** $\mathcal{D}_L$ and a **test set** $\mathcal{D}_t$, and $\mathcal{D}_L$ further into a **training set** $\mathcal{D}_T$ and a **validation set** $\mathcal{D}_V$. **Hyper-parameter tuning** then goes as follows:

1. learn the model on $\mathcal{D}_T$;
2. evaluate it on $\mathcal{D}_V$ for each candidate k and keep the best one;
3. report the final performance on $\mathcal{D}_t$.

In practice one plots the validation accuracy against k and takes the maximiser. Cross-validation and further good practice for model selection follow with decision trees (section 2.3).

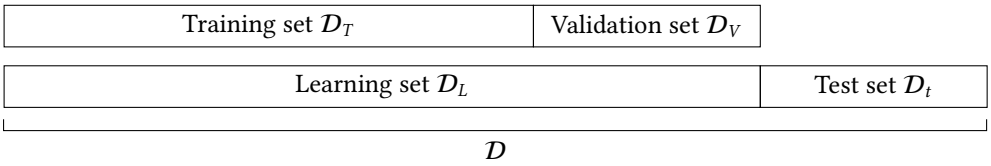
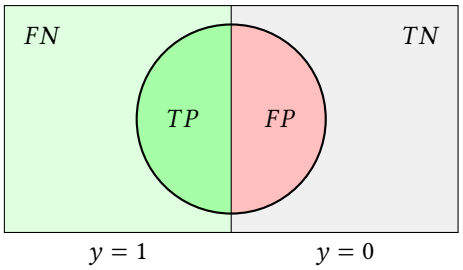


Figure 2: Splitting the dataset $\mathcal{D}$ for hyper-parameter tuning.

Binary classification metrics. For $y \in \{0, 1\}$, the predictions on a labelled set are counted in the **confusion table** (table 1 and fig. 3): **true positives** TP and **true negatives** TN are the correct predictions, **false positives** FP and **false negatives** FN the wrong ones.

	True condition	
Predicted	$y = 1$	$y = 0$
$\hat{y} = 1$	TP	FP
$\hat{y} = 0$	FN	TN

Table 1: Confusion table of a binary classifier.



disc: predicted $\hat{y} = 1$

$$\text{prec} = \frac{TP}{\text{disc}}, \quad \text{rec} = \frac{TP}{\text{left half}}$$

Figure 3: The four counts as regions: positives on the left, negatives on the right.

Definition 1.3 (Classification metrics)**.**

$$\text{accuracy:} \quad \text{acc} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{F1 score:} \quad f1 = \frac{2 \cdot \text{prec} \cdot \text{rec}}{\text{prec} + \text{rec}},$$

$$\text{precision:} \quad \text{prec} = \frac{TP}{TP + FP}, \qquad \text{sensitivity/recall:} \quad \text{rec} = \frac{TP}{TP + FN},$$

$$\text{specificity:} \quad \text{tnr} = \frac{TN}{FP + TN}, \qquad \text{false negative rate:} \quad \text{fnr} = \frac{FN}{TP + FN},$$

$$\text{false positive rate:} \quad \text{fpr} = \frac{FP}{FP + TN}.$$

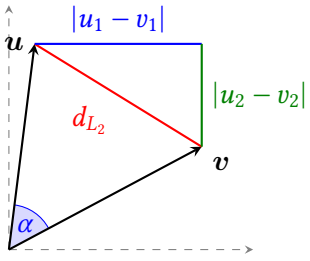
The F1 score is the harmonic mean of precision and recall, $f1 = 2\Big/\left(\frac{1}{\text{prec}} + \frac{1}{\text{rec}}\right)$.

Precision and recall trade off: increasing one most often decreases the other. All these numbers must be read with care when the classes are imbalanced.

1.3 Distances, scaling and dimensionality

Distance measures. k-NN works with any distance measure, which makes it highly flexible. For $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^D$ (fig. 4):

- **Euclidean** (L_2) distance $\|\boldsymbol{u} - \boldsymbol{v}\|_2 = \sqrt{\sum_j (u_j - v_j)^2}$;
- L_1 distance $\sum_j |u_j - v_j|$ and L_∞ distance $\max_j |u_j - v_j|$;
- the **angle** α between them, $\cos\alpha = \dfrac{\boldsymbol{u}^T \boldsymbol{v}}{\|\boldsymbol{u}\|\|\boldsymbol{v}\|}$;
- the **Mahalanobis distance** $\sqrt{(\boldsymbol{u} - \boldsymbol{v})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{u} - \boldsymbol{v})}$, with $\boldsymbol{\Sigma}$ symmetric positive definite;
- Hamming distance, edit distance, and others.



$$d_{L_1} = |u_1 - v_1| + |u_2 - v_2|$$
$$d_{L_2} = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2}$$
$$d_{L_\infty} = \max_j |u_j - v_j| = |u_1 - v_1|$$

Figure 4: L_1, L_2, L_∞ distances and angle α between two vectors of $\mathbb{R}^2$.

Scaling. A distance mixes all features, so features on very different scales distort it. For instance, if in $\mathbb{R}^2$ the feature x_1 is of the order of metres and x_2 of the order of centimetres (say x_1 up to 1000 cm, x_2 below 6 cm), the distance, hence the nearest neighbour, is determined almost only by x_1: the 1-NN decision regions become stripes orthogonal to the x_1-axis and x_2 is effectively ignored. Two remedies:

- **standardisation**: scale each feature j to zero mean and unit variance, $x_{j,\text{std}} = \dfrac{x_j - \mu_j}{\sigma_j}$, with μ_j, σ_j the mean and standard deviation of feature j; this is standard practice in machine learning, since many models are sensitive to differences in scale;
- the Mahalanobis distance with $\boldsymbol{\Sigma} = \text{diag}(\sigma_1^2, \dots, \sigma_D^2)$, which equals the Euclidean distance between the standardised data.

Indeed $\boldsymbol{\Sigma}^{-1} = \text{diag}(\sigma_1^{-2}, \dots, \sigma_D^{-2})$ and the means cancel in the differences, so

$$(\boldsymbol{u} - \boldsymbol{v})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{u} - \boldsymbol{v}) = \sum_{j=1}^{D} \frac{(u_j - v_j)^2}{\sigma_j^2} = \sum_{j=1}^{D} \left(\frac{u_j - \mu_j}{\sigma_j} - \frac{v_j - \mu_j}{\sigma_j} \right)^2 = \|\boldsymbol{u}_{\text{std}} - \boldsymbol{v}_{\text{std}}\|_2^2.$$

Curse of dimensionality. Draw $N = 20$ samples uniformly from the discrete input space $\{1, \dots, 10\}^D$, which has 10^D points; the samples occupy at most 20 of them. For $D = 1$ they cover most or all of the 10 points, for $D = 2$ at most 20% of the 100 points (about 18%, since some samples coincide), and for $D = 3$ at most 2% of the 1000 points. Hence in high dimension the nearest neighbour of a new point is typically far away, and to cover the same fraction of the input space N has to grow exponentially with the number of features: this is the **curse of dimensionality**, to keep in mind when using k-NN on high-dimensional data.

Cost. Naive k-NN is expensive: memory and inference are both $O(N)$ (for fixed D), since the whole training set is stored and every query is compared with all training points. Tree-based search structures such as the **k-d tree** give efficient (approximate) nearest-neighbour search, at the price of an additional computation performed only once (building the structure).

2 Decision Trees

2.1 Decision trees and inference

Structure. A decision tree classifies by asking a sequence of questions, as in the game of twenty questions: "does it have fur?", if yes "does it bark?" (dog or cat), if no "does it fly?" (bird or fish). In a **decision tree** each inner **node** is a test on a feature, each **branch** leaving it is one outcome of the test, and each **leaf** is a region of the input space together with the distribution of the training labels in that region. Figure 5 shows a tree for the tennis dataset, whose goal is to predict *PlayTennis* for unseen days: all four overcast days are "Yes", so that branch is a leaf right away, while sunny days need a test on Humidity and rainy days one on Windy. The tree classifies all 14 training days correctly.

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

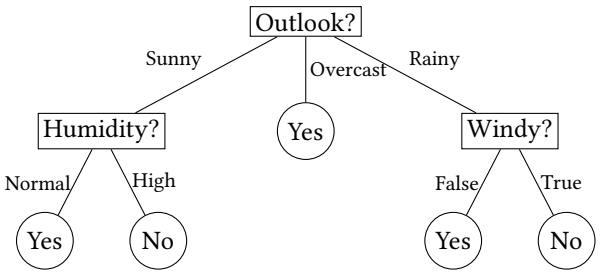


Figure 5: Tennis dataset and a decision tree that classifies it without errors.

Numerical features. For $\boldsymbol{x} \in \mathbb{R}^D$ the simplest test is a **binary split** on a single feature, $x_j \leq a$ (true) or $x_j > a$ (false); in this lecture these are the only tests on numerical features. Each split cuts a region with an axis-parallel hyperplane, so a decision tree partitions the input space into cuboid (axis-aligned box) regions, one per leaf (fig. 6).

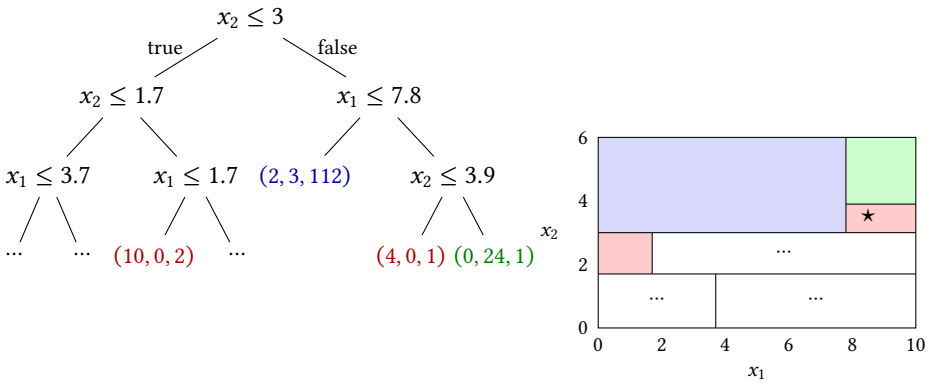


Figure 6: Top of a tree on three classes (leaf counts (n_1, n_2, n_3), unexpanded subtrees as $\cdots$) and its partition of $[0, 10] \times [0, 6]$; the star is $\boldsymbol{x} = (8.5, 3.5)^T$.

Definition 2.1 (Inference in a decision tree)**.** Let the classes be $\{1, \dots, C\}$. To classify $\boldsymbol{x}$, test its features from the root down to the leaf whose region $\mathcal{R}$ contains $\boldsymbol{x}$, and let $\boldsymbol{n}_\mathcal{R} = (n_{1,\mathcal{R}}, \dots, n_{C,\mathcal{R}})$ be the class counts of the training points in $\mathcal{R}$. Then

$$p(y = c \mid \mathcal{R}) = \frac{n_{c,\mathcal{R}}}{\sum_{c'=1}^{C} n_{c',\mathcal{R}}}, \qquad \hat{y} = \arg\max_c \; p(y = c \mid \boldsymbol{x}) = \arg\max_c \; p(y = c \mid \mathcal{R}) = \arg\max_c \; n_{c,\mathcal{R}},$$

i.e. $\boldsymbol{x}$ gets the majority label of its region, as in k-NN (definition 1.1).

Example 2.2. In fig. 6, $\boldsymbol{x} = (8.5, 3.5)^T$ has $x_2 > 3$, then $x_1 > 7.8$, then $x_2 \leq 3.9$, and reaches the leaf with counts $(4, 0, 1)$. Hence $p(y \mid \boldsymbol{x}) = (\frac{4}{5}, 0, \frac{1}{5})$ and $\hat{y} = 1$.

2.2 Growing a tree: splitting criteria

The optimal tree is intractable. As for k-NN, the goal is generalisation, and the tree is chosen with the split of fig. 2: build trees on $\mathcal{D}_T$, predict the labels of $\mathcal{D}_V$, keep the tree with the best validation performance and report its performance on $\mathcal{D}_t$. The naive idea is to build all possible trees. Already the tests are many: a feature with n distinct values in the training set has $n - 1$ thresholds a that split the data differently (one between each pair of consecutive values), so 2 features with 300 distinct values each give $2 \cdot 299 = 598$ possible tests at the root, slightly fewer at each descendant. The number of trees explodes, and finding the optimal tree is NP-complete (optimal meaning that it minimises the expected number of tests needed to classify an unknown sample; even identifying the root node of an optimal tree is NP-hard). Instead, the tree is grown top-down on the training data, choosing the best split node by node with a **greedy heuristic**.

Definition 2.3 (Impurity improvement)**.** Let $i(t)$ be a splitting criterion measuring the impurity of the class distribution $\pi_c = p(y = c \mid t)$ of the training points at node t. A split s sends a fraction p_L of them to the left child t_L and $p_R = 1 - p_L$ to the right child t_R. The **improvement** of s is

$$\Delta i(s, t) = i(t) - p_L \, i(t_L) - p_R \, i(t_R).$$

At each node the tree is split with the test (over all features and thresholds) of largest $\Delta i(s, t)$.

Misclassification rate. The first candidate is the **misclassification rate** $i_E(t) = 1 - \max_c \pi_c$, the training error of the node when it predicts its majority class. For instance, a node with 150 red, 100 blue and 50 green points has $i_E(t) = 1 - \frac{150}{300} = 0.5$; the split $x_1 \le 5$ of fig. 7(a) leaves a pure left child and a right child with $i_E(t_R) = 1 - \frac{100}{150} = \frac{1}{3}$, so $\Delta i_E = 0.5 - \frac{1}{2} \cdot 0 - \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{3}$. The misclassification rate has two problems:

1. *no split may be performed although two splits together would classify perfectly.* In fig. 7(b), 40 of 200 points are red, so $i_E(t) = \frac{40}{200}$. The split $x_1 \le 5$ gives a pure child and a child with 40 red out of 100: $p_L\, i_E(t_L) + p_R\, i_E(t_R) = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot \frac{40}{100} = \frac{40}{200}$, and the same holds for $x_2 \le 3$. Hence $\Delta i_E = 0$ for both splits and none is made, although $x_1 > 5$ and $x_2 \le 3$ together isolate the red class;
2. *it is insensitive to changes in the class probabilities.* Starting from class counts $(400, 400)$, so $i_E(t) = \frac{1}{2}$, the split a into $(100, 300)$ and $(300, 100)$ and the split b into $(200, 400)$ and $(200, 0)$ have the same improvement,

$$\Delta i_E(a, t) = \tfrac{1}{2} - \tfrac{1}{2} \cdot \tfrac{1}{4} - \tfrac{1}{2} \cdot \tfrac{1}{4} = 0.25, \qquad \Delta i_E(b, t) = \tfrac{1}{2} - \tfrac{3}{4} \cdot \tfrac{1}{3} - \tfrac{1}{4} \cdot 0 = 0.25,$$

   although b produces a pure node.

On the data of fig. 6 the same flaw shows up: as a function of the threshold, Δi_E is exactly 0 for most thresholds of x_1 and x_2, and positive (at most about 0.04) only for x_1 between about 6.5 and 10 and for x_2 close to 3.

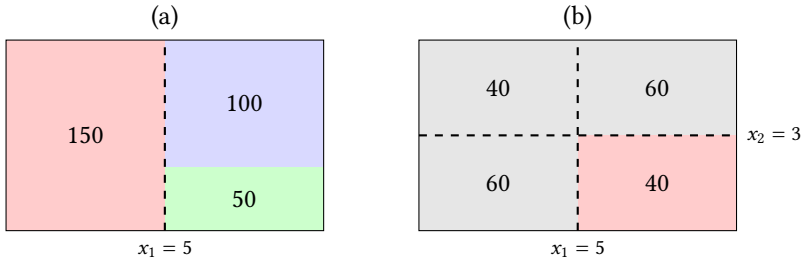


Figure 7: Class counts of two nodes: (a) a useful split $x_1 \le 5$; (b) data where neither $x_1 \le 5$ nor $x_2 \le 3$ improves i_E.

Impurity measures. A suitable criterion $i(t)$ should be *maximal* when the classes are equally distributed in the node, *minimal* (usually 0) when the node is pure, and *symmetric* in the classes.

Definition 2.4 (Impurity measures)**.** With $\pi_c = p(y = c \mid t)$:

$$i_E(t) = 1 - \max_c \pi_c, \qquad i_H(t) = -\sum_{c=1}^{C} \pi_c \log_2 \pi_c, \qquad i_G(t) = \sum_{c=1}^{C} \pi_c(1 - \pi_c) = 1 - \sum_{c=1}^{C} \pi_c^2,$$

the **misclassification rate**, the **entropy** (with $0 \log_2 0 = 0$, the limit of $x \log x$ as $x \to 0^+$) and the **Gini index**. The two forms of i_G agree because $\sum_c \pi_c(1 - \pi_c) = \sum_c \pi_c - \sum_c \pi_c^2$ and $\sum_c \pi_c = 1$.

For two classes, with $\pi = \pi_1$ and $\pi_2 = 1 - \pi$ (fig. 8),

$$i_E = 1 - \max(\pi, 1 - \pi) = \min(\pi, 1 - \pi), \qquad i_G = 1 - \pi^2 - (1 - \pi)^2 = 2\pi(1 - \pi),$$
$$i_H = -\pi \log_2 \pi - (1 - \pi) \log_2 (1 - \pi) \ :$$

all three vanish at $\pi \in \{0, 1\}$ and are maximal at $\pi = \frac{1}{2}$, with maximum $\frac{1}{2}$ for i_E, i_G and 1 for i_H.

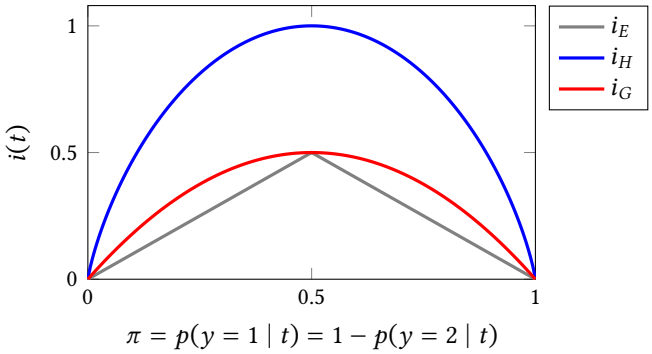


Figure 8: Misclassification rate, entropy and Gini index for two classes.

Remark. Entropy and Gini index are strictly concave in π, while i_E is linear as long as the majority class does not change; this is why they fix both problems. With i_G, in (b) of fig. 7 either split has $\Delta i_G > 0$, and b is preferred to a:

$$i_G(t) = 1 - 0.8^2 - 0.2^2 = 0.32, \qquad \Delta i_G = 0.32 - \tfrac{1}{2} \cdot 0 - \tfrac{1}{2}\,(1 - 0.6^2 - 0.4^2) = 0.08,$$
$$\Delta i_G(a, t) = \tfrac{1}{2} - \tfrac{3}{8} = 0.125 \;<\; \Delta i_G(b, t) = \tfrac{1}{2} - \tfrac{3}{4} \cdot \tfrac{4}{9} = \tfrac{1}{6}.$$

Entropy and information. For a discrete random variable X with values $\{x_1, \dots, x_n\}$ the **Shannon entropy** is

$$\mathbb{H}(X) = -\sum_{i=1}^{n} p(X = x_i) \log_2 p(X = x_i),$$

the expected number of bits needed to encode a randomly drawn value of X under the most efficient code. For a variable with 4 values it is 2 bits for the uniform distribution ($-4 \cdot \frac{1}{4} \log_2 \frac{1}{4}$), 0 when one value has probability 1, and in between (1.9333 in one non-uniform case) otherwise: higher entropy means a flatter histogram and less predictable values, lower entropy a peakier histogram and more predictable values. As an example of efficient coding, take four messages with probabilities 0.01 ("there is free beer"), 0.02 ("you have an exam"), 0.30 ("you have a lecture") and 0.67 ("nothing happening"). The **Huffman code** gives shorter codewords to likelier messages, here 111, 110, 10, 0, with average length

$$0.01 \cdot 3 + 0.02 \cdot 3 + 0.30 \cdot 2 + 0.67 \cdot 1 = 1.36 \text{ bits}.$$

Remark. In general $\mathbb{H}(X)$ is a lower bound on the average code length, reached when all probabilities are powers of $\frac{1}{2}$; here $\mathbb{H} \approx 1.09 < 1.36$.

Gini index. $i_G(t) = \sum_c \pi_c(1 - \pi_c)$ is the probability that a randomly chosen instance of the node is misclassified when it is labelled at random according to the node's class distribution: π_c is the probability of picking an instance of class c, and $1 - \pi_c$ the probability that its random label is wrong. In practice the choice between entropy and Gini index matters in only about 2% of the cases (Raileanu and Stoffel); the Gini index has the small advantage of needing no logarithm, so it can be a bit faster to compute.

Example 2.5. The data of fig. 6 have 300 points with class counts $(67, 65, 168)$, so at the root

$$i_G(t) = 1 - \left(\frac{67}{300}\right)^2 - \left(\frac{65}{300}\right)^2 - \left(\frac{168}{300}\right)^2 \approx 0.5896.$$

The test $x_2 \le 3$ sends 153 points to t_L, with counts $(61, 38, 54)$ and $i_G(t_L) \approx 0.6548$, and 147 to t_R, with counts $(6, 27, 114)$ and $i_G(t_R) \approx 0.3632$. Hence

$$\Delta i_G(x_2 \le 3, t) = i_G(t) - \frac{153}{300}\, i_G(t_L) - \frac{147}{300}\, i_G(t_R) \approx 0.07768,$$

the largest improvement over all tests, so $x_2 \le 3$ becomes the root. As a function of the threshold, Δi_G is positive and varies smoothly for all thresholds of both features (up to about 0.035 for x_1), unlike Δi_E.

Growing the tree. Applying the greedy step recursively, the label distributions of the nodes become progressively purer (mostly instances of one class); fig. 6 is the top of the tree obtained in this way with the Gini index. On the training data the accuracy of the tree grows with its depth: 58.3% at depth 1, 77% at depth 2, 84.3% at depth 3, 90.3% at depth 4, and 100% for the **maximally pure** tree, whose leaves are all pure and whose decision regions include many small boxes around single points. Perfect training accuracy does not mean good generalisation.

2.3 Overfitting, pruning and extensions

Overfitting. **Overfitting** typically occurs when the model tries to fit the training data perfectly, and means poor generalisation. It is recognised by a low training error (possibly 0) together with a comparably high validation error. For decision trees, training accuracy never decreases with a split: since the class counts of t are the sums of those of its children, $n_{c,t} = n_{c,t_L} + n_{c,t_R}$, the number of correctly classified training points can only grow,

$$\max_c n_{c,t} \leq \max_c n_{c,t_L} + \max_c n_{c,t_R}.$$

The validation accuracy tells how well the model generalises. In a typical experiment, as the number of leaves grows from 2 to about 35, the training accuracy rises steadily from about 0.6 to 1, while the validation accuracy rises with it at first (**underfitting** with very few leaves), peaks at about 0.88 with 16 leaves and then drops to about 0.77 (overfitting). In practice, with increasing model size, data size or training time, the performance sometimes first improves, then gets worse, and then improves again: this **double descent phenomenon** was observed recently and is actively studied.

Model selection. Model selection and the fight against overfitting use the split of fig. 2: models are built on the training set and compared on the validation set, and the test set is touched only once, at the end, to report the final performance.

Definition 2.6 (K-fold cross-validation)**.** Split the learning set $\mathcal{D}_L$ into K disjoint folds $\mathcal{D}_{V_1}, \ldots, \mathcal{D}_{V_K}$ ($K = 10$ is common) and let $\mathcal{D}_{T_k} = \mathcal{D}_L \setminus \mathcal{D}_{V_k}$ (fig. 9). For each k, train on the $K-1$ folds $\mathcal{D}_{T_k}$ and evaluate on $\mathcal{D}_{V_k}$; the average over the folds,

$$\mathrm{err}_{\mathrm{CV}} = \frac{1}{K} \sum_{k=1}^{K} \mathrm{err}_{\mathcal{D}_{V_k}}(f_k), \qquad f_k \text{ trained on } \mathcal{D}_{T_k},$$

estimates the error of a setting of the hyper-parameters (or of the chosen model, for model selection). Repeat for different settings of the hyper-parameters, then train with the best setting on all of $\mathcal{D}_L$ and test on $\mathcal{D}_t$.

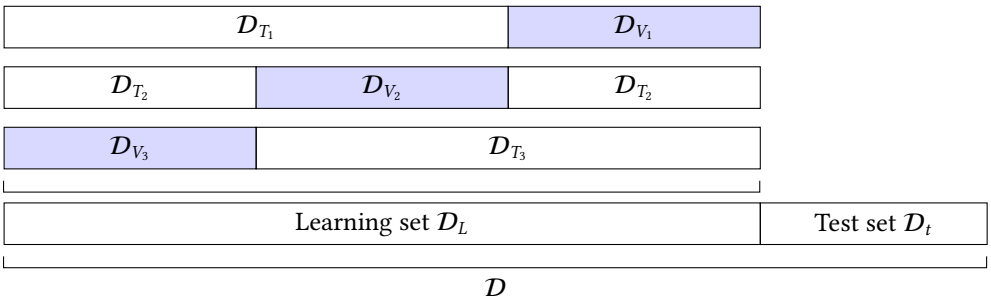


Figure 9: 3-fold cross-validation on the learning set; validation folds shaded.

In **leave-one-out cross-validation** (LOOCV) each model is trained on all but one sample: with N samples it is N-fold cross-validation. It is useful when data are scarce, since it uses as much data as

possible for training and still gives a good estimate of the performance, but the model has to be trained N times. With large amounts of data and an expensive model, a smaller K or a single validation set is the better choice.

Stopping criteria. The tree is grown by recursive splitting, which must stop at some point. Possible **stopping** (or **pre-pruning**) criteria: the distribution in the branch is pure, $i(t) = 0$; a maximum depth is reached; the number of samples in each branch falls below a threshold t_n; the benefit of splitting falls below a threshold, $\Delta i(s, t) < t_\Delta$; the accuracy on the validation set. Alternatively, grow the tree maximally and then (post-)prune it.

Definition 2.7 (Reduced error pruning)**.** Let T be a decision tree, t an inner node and T_t the subtree rooted at t. **Pruning** T w.r.t. t deletes all descendants of t (but not t, which becomes a leaf); the pruned tree is written $T \setminus T_t$ (fig. 10). **Reduced error pruning** uses the error $\mathrm{err}_{D_V}(T)$ on the validation set:

1. for each inner node t compute $\mathrm{err}_{D_V}(T \setminus T_t)$;
2. prune T at the node that yields the largest error reduction;
3. repeat until $\mathrm{err}_{D_V}(T) < \mathrm{err}_{D_V}(T \setminus T_t)$ for all inner nodes t.

After pruning, both training and validation data may be used to update the labels at each leaf.

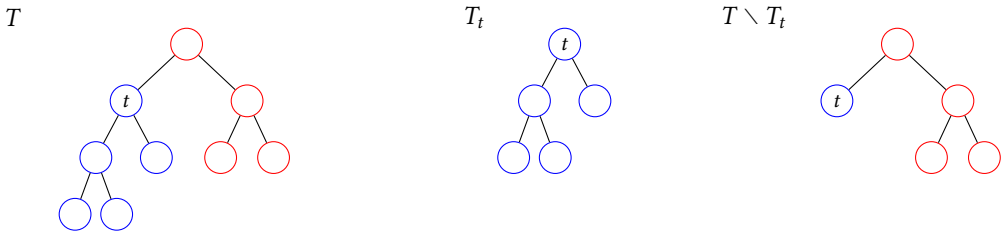


Figure 10: A tree T, its subtree T_t and the pruned tree $T \setminus T_t$.

Categorical features and regression. Categorical features can be tested by equality, e.g. "Outlook = overcast?" (yes: play), otherwise "Temperature = hot?" (yes: do not play), otherwise "Wind = weak?", and so on, or by one branch per value as in fig. 5; the algorithm variants ID3, C4.5 and CART handle them differently. For **regression** ($y_i \in \mathbb{R}$), a leaf predicts the mean of the training outputs in its region instead of the mode, and the splitting heuristic is the mean squared error, i.e. definition 2.3 with $i(t) = \frac{1}{n_t} \sum_{i \in t} (y_i - \bar{y}_t)^2$, where n_t is the number of training points at t and $\bar{y}_t$ their mean output. The prediction is then a piecewise constant function of $\boldsymbol{x}$; on one-dimensional data a tree of depth 5 follows the points much more closely than one of depth 2.

Properties. Decision trees are human interpretable and handle any combination of numerical and categorical features and targets; their extensions (random forests, boosted trees) have very competitive performance, e.g. in Kaggle competitions. Compared to k-NN (section 1) they have a much better complexity in memory/storage and inference, since only the tree is stored and a query follows a single root-to-leaf path, and a more flexible decision function.

Ensembles. An **ensemble** aggregates the predictions of many (diverse) classifiers to improve the performance; the main benefit is a lower variance of the model, obtained by averaging (see the bias-variance trade-off in the lecture on linear regression). Main types:

- **bagging** (bootstrap aggregating): create new datasets by sampling the training set with replacement, train a separate classifier on each and combine their predictions, e.g. by average or majority vote;
- **boosting**: train (weak) classifiers incrementally, each one correcting the previous mistakes by giving higher weight to hard (misclassified) examples;

- **stacking**: train a meta-classifier that takes the base classifiers' predictions as features;
- further approaches: bucket of models, Bayesian model averaging, Bayes optimal classifier.

Random forests. Bagged decision trees can be highly correlated, which reduces the benefit of bagging since it needs diverse classifiers. A **random forest** therefore learns each tree on a random subset of the features as well, i.e. bagging at instance level plus bagging at feature level. Random forests are widely used because of their good "out-of-the-box" performance (not much tuning required), but are less interpretable than a single tree. Rule of thumb for the number of random features: $\log_2 D$ for regression and $\sqrt{D}$ for classification, with D the number of features.

Boosting: AdaBoost and XGBoost. Boosting trains "weak learners" incrementally, e.g. **decision stumps** (one-level trees):

1. initialise a weight vector over the training examples with uniform weights;
2. loop: train a weak learner on the weighted examples (e.g. by weighted sampling), then increase the weights of the misclassified examples;
3. predict by the (error-based) weighted majority of the weak learners.

Adaptive boosting (**AdaBoost**) and gradient boosting (**XGBoost**) differ mainly in how the weights are updated: AdaBoost uses closed-form weight updates based on the errors, XGBoost adds one tree, and one level in the tree, at a time, splitting greedily based on the gradient w.r.t. a (custom) loss function.